

Pokémon Chronicles: Pocket Beasts

Documentación del Proyecto



Autor: [Marciuc Dragos Stefan]

Índice

1. Descripción del Proyecto	2
1.1. Concepto y Jugabilidad	2
1.2. Características principales	2
1.3. Entorno y Diseño	2
1.4. Mecánicas de Juego	2
1.5. Sistema de Progresión	2
1.6. Detalles Técnicos	3
2. Justificación del Proyecto	3
2.1. Interés Tecnológico	3
2.2. Selección del Género	3
2.3. Valor Formativo	3
3. Alcance del Proyecto	3
4. Alternativas Existentes	4
4.1. Pokémon Oficiales (Nintendo)	4
4.2. Pokémon Showdown	4
4.3. Fangames Varios	4
5. Stack Tecnológico	4
6. Objetivos	5
6.1. Objetivos Principales	5
6.2. Objetivos Secundarios	5
7. Desarrollo del Videojuego	5
7.1. Forma y Mecánica	5
7.2. Mecánicas Principales	6
7.3. Target	6
8. Requisitos del Sistema	6
8.1. Requisitos Funcionales	6
8.2. Requisitos No Funcionales	7
8.3. Requisitos de Interfaz	7
9. Casos de Uso	7
9.1. CU-01: Iniciar Combate	7
9.2. CU-02: Capturar Criatura	8
9.3. CU-03: Gestionar Equipo	8
10. Modelo y Diseño de la Base de Datos	9
10.1. Esquema SQLite	9
10.2. Modelo de Datos en C#	10
10.3. Relaciones Principales	10

1 Descripción del Proyecto

1.1 Concepto y Jugabilidad

“Pokémon Chronicles: Pocket Beasts” es un videojuego desarrollado en Unity que recrea las mecánicas clásicas de la saga Pokémon. El jugador podrá explorar un mundo, capturar criaturas y participar en combates por turnos, implementando un RPG funcional con sistemas completos de progresión, combate y gestión de datos.

1.2 Características principales

- **Exploración del mundo:** Movimiento libre por diferentes zonas con encuentros aleatorios
- **Sistema de combate:** Combates por turnos con IA básica enemiga
- **Captura de criaturas:** Sistema de captura con diferentes probabilidades de éxito
- **Progresión:** Sistema de experiencia, niveles y evolución de criaturas
- **Base de datos:** Gestión de datos de criaturas, movimientos y tipos
- **Persistencia:** Guardado y carga de partidas mediante SQLite

1.3 Entorno y Diseño

El juego presenta las siguientes características:

- **Estilo visual:** Gráficos 2D con perspectiva cenital en estilo pixel art retro
- **Interfaz:** HUD de combate e inventario intuitiva y clara
- **Sonido:** Efectos de audio para acciones principales y música ambiental

1.4 Mecánicas de Juego

El sistema de juego está construido sobre Unity, ofreciendo:

Acción	Implementación
Movimiento	Teclas WASD o flechas direccionales
Interacción	Tecla E o clic
Menú	Tecla ESC o botón específico
Combate	Sistema por turnos

Cuadro 1: Controles básicos del juego

1.5 Sistema de Progresión

El juego incluye un sistema de progresión basado en:

- **Experiencia:** Ganada tras ganar combates
- **Niveles:** Subida de nivel con mejora de estadísticas
- **Evoluciones:** Transformación de criaturas al alcanzar niveles específicos

1.6 Detalles Técnicos

El proyecto utiliza:

- **Motor:** Unity 2022.3 LTS
- **Lenguaje:** C#
- **Base de datos:** SQLite para persistencia local
- **Control de versiones:** Git + GitHub

2 Justificación del Proyecto

2.1 Interés Tecnológico

El desarrollo de videojuegos es un campo en constante evolución que combina programación, diseño y creatividad. Este proyecto permite aplicar conocimientos de desarrollo de software en un contexto práctico y atractivo.

2.2 Selección del Género

¿Y por qué un RPG al estilo Pokémon?

- **Compatibilidad técnica:** Género bien documentado con mecánicas establecidas
- **Curva de aprendizaje:** Permite implementar sistemas progresivamente
- **Valor educativo:** Abarca múltiples áreas del desarrollo de software

2.3 Valor Formativo

Este proyecto ofrece:

- Desarrollo de habilidades en C# y Unity
- Implementación de arquitecturas de software
- Gestión de bases de datos y persistencia
- Control de versiones y metodologías ágiles

3 Alcance del Proyecto

El juego incluirá:

- Un mundo explorable con diferentes zonas
- Sistema de combate por turnos funcional
- En un principio 8-10 criaturas únicas con sus evoluciones
- Sistema de captura y gestión de equipo

- Persistencia de datos con SQLite
- Interfaz de usuario completa

Nota: Los elementos finales podrán ajustarse durante el desarrollo según pruebas técnicas.

4 Alternativas Existentes

4.1 Pokémon Oficiales (Nintendo)

- **Ventajas:** Juegos completos y pulidos
- **Desventajas:** Código cerrado, no permiten modificación
- **Diferenciación:** Nuestro proyecto es educativo y de código abierto

4.2 Pokémon Showdown

- **Ventajas:** Sistema de combate muy desarrollado
- **Desventajas:** Solo combates, sin exploración
- **Diferenciación:** Se ofrece experiencia RPG completa

4.3 Fangames Varios

- **Ventajas:** Gran variedad de ideas y enfoques
- **Desventajas:** Calidad variable, pocos documentados
- **Diferenciación:** Enfoque en arquitectura técnica y documentación

5 Stack Tecnológico

Tecnología	Uso	Versión
Unity	Motor principal del juego	2022.3 LTS
C#	Lenguaje de programación	.NET 4.x
SQLite	Base de datos relacional	3.x
Proxmox VE	Virtualización y contenedores	8.x
LXC Container	Contenedor Linux para servicios	Ubuntu 22.04
Git + GitHub	Control de versiones	Latest
Aseprite o variantes	Creación de sprites	Latest

Cuadro 2: Stack tecnológico del proyecto

6 Objetivos

6.1 Objetivos Principales

Los objetivos fundamentales de este proyecto son:

- **Desarrollo del juego funcional:** Crear un RPG que permita:
 - Exploración del mundo con movimiento libre
 - Sistema de combate por turnos funcional
 - Captura y gestión de criaturas
- **Sistema de base de datos:** Implementar persistencia que incluya:
 - Datos de criaturas y movimientos
 - Guardado de partidas de usuario
 - Gestión de progreso del jugador
- **Interfaz de usuario:** Desarrollar interfaces que ofrezcan:
 - HUD de combate clara e intuitiva
 - Menús de gestión de equipo
 - Sistema de navegación fluido

6.2 Objetivos Secundarios

Como metas complementarias se plantean:

- Implementar sistema de evoluciones
- Crear IA básica para combates
- Desarrollar efectos visuales y sonoros

Nota: Estos objetivos podrán ajustarse durante el desarrollo en función de los resultados obtenidos en las primeras pruebas técnicas.

7 Desarrollo del Videojuego

7.1 Forma y Mecánica

- **Género:** RPG por turnos
- **Perspectiva:** 2D cenital
- **Estilo visual:** Pixel art retro
- **Estructura:** Mundo abierto con progresión guiada

7.2 Mecánicas Principales

- **Exploración:**
 - Movimiento libre por el mundo
 - Encuentros aleatorios en hierba
 - Interacción con NPCs y objetos
- **Combate:**
 - Sistema por turnos basado en stats
 - Tipos elementales con ventajas/desventajas
 - 4 movimientos por criatura
- **Progresión:**
 - Sistema de experiencia y niveles
 - Evoluciones en niveles específicos
 - Mejora de stats por nivel

7.3 Target

- Jugadores de 12-35 años
- Fanáticos de RPGs y juegos retro
- Sesiones de 15-45 minutos

8 Requisitos del Sistema

8.1 Requisitos Funcionales

- **RF-01:** Sistema de exploración del mundo
 - Movimiento del jugador por el mapa
 - Encuentros aleatorios con criaturas
 - Transición entre zonas
- **RF-02:** Sistema de combate por turnos
 - Selección de movimientos
 - Cálculo de daño y efectos
 - IA básica para oponentes
- **RF-03:** Captura y gestión de criaturas
 - Sistema de captura con probabilidades
 - Gestión de equipo (6 criaturas máximo)
 - Almacenamiento en PC

- **RF-04:** Progresión y evoluciones
 - Sistema de experiencia y niveles
 - Evolución en niveles específicos
 - Mejora de estadísticas
- **RF-05:** Persistencia de datos
 - Guardado de partidas en SQLite
 - Carga de partidas guardadas
 - Gestión de múltiples usuarios

8.2 Requisitos No Funcionales

- **RNF-01:** Rendimiento
 - 30 FPS en hardware estándar
 - Tiempos de carga menores a 3 segundos
- **RNF-02:** Usabilidad
 - Interfaz intuitiva y clara
 - Curva de aprendizaje suave
 - Feedback visual inmediato
- **RNF-03:** Compatibilidad
 - Windows 10/11
 - Resoluciones 1080p y 720p

8.3 Requisitos de Interfaz

- **RI-01:** HUD de combate
 - Información clara de HP y estado
 - Selección intuitiva de movimientos
 - Indicadores de turno
- **RI-02:** Menús del juego
 - Navegación con teclado/ratón
 - Diseño consistente en todas las pantallas
 - Acceso rápido a funciones principales

9 Casos de Uso

9.1 CU-01: Iniciar Combate

Actor Principal: Jugador

Precondiciones:

- Jugador explorando el mundo
- En zona de hierba o encuentro forzado

Flujo Principal:

1. Sistema detecta encuentro aleatorio
2. Transición a escena de combate
3. Mostrar HUD de combate con opciones
4. Iniciar sistema de turnos
5. Jugador selecciona movimiento
6. Resolver turno y mostrar resultados

9.2 CU-02: Capturar Criatura

Actor Principal: Jugador

Precondiciones:

- En combate con criatura salvaje
- Criatura con vida reducida
- Tener Pokéballs en inventario

Flujo Principal:

1. Jugador selecciona “Capturar”
2. Sistema calcula probabilidad de éxito
3. Mostrar animación de lanzamiento
4. Determinar resultado de captura
5. Actualizar equipo o inventario

9.3 CU-03: Gestionar Equipo

Actor Principal: Jugador

Precondiciones:

- En menú principal o centro Pokémon
- Tener criaturas en equipo o PC

Flujo Principal:

1. Acceder a menú de equipo
2. Ver estadísticas de criaturas
3. Reorganizar orden del equipo
4. Intercambiar entre equipo y PC
5. Guardar cambios automáticamente

10 Modelo y Diseño de la Base de Datos

10.1 Esquema SQLite

```
1 -- Tabla de usuarios
2 CREATE TABLE Users (
3     UserID INTEGER PRIMARY KEY AUTOINCREMENT,
4     Username VARCHAR(50) UNIQUE NOT NULL,
5     PasswordHash VARCHAR(255) NOT NULL,
6     CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP
7 );
8
9 -- Tabla de criaturas base
10 CREATE TABLE Creatures (
11     CreatureID INTEGER PRIMARY KEY,
12     Name VARCHAR(50) NOT NULL,
13     Type1 VARCHAR(20) NOT NULL,
14     Type2 VARCHAR(20),
15     BaseHP INTEGER NOT NULL,
16     BaseAttack INTEGER NOT NULL,
17     BaseDefense INTEGER NOT NULL,
18     BaseSpeed INTEGER NOT NULL,
19     EvolutionLevel INTEGER,
20     EvolvesTo INTEGER
21 );
22
23 -- Tabla de movimientos
24 CREATE TABLE Moves (
25     MoveID INTEGER PRIMARY KEY,
26     Name VARCHAR(50) NOT NULL,
27     Type VARCHAR(20) NOT NULL,
28     Power INTEGER,
29     Accuracy INTEGER,
30     PP INTEGER NOT NULL
31 );
32
33 -- Tabla de partidas guardadas
34 CREATE TABLE SaveGames (
35     SaveID INTEGER PRIMARY KEY AUTOINCREMENT,
36     UserID INTEGER,
37     PlayerData TEXT NOT NULL,
38     CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
39     FOREIGN KEY (UserID) REFERENCES Users(UserID)
40 );
```

10.2 Modelo de Datos en C#

```
1 [System.Serializable]
2 public class Creature
3 {
4     public int CreatureID;
5     public string Name;
6     public int Level;
7     public int CurrentHP;
8     public int MaxHP;
9     public int Attack;
10    public int Defense;
11    public int Speed;
12    public int Experience;
13    public List<Move> Moves;
14    public CreatureType Type1;
15    public CreatureType Type2;
16 }
17
18 [System.Serializable]
19 public class PlayerData
20 {
21     public string PlayerName;
22     public Vector2 Position;
23     public List<Creature> Team;
24     public List<Item> Inventory;
25     public int Money;
26 }
27
28 public enum CreatureType
29 {
30     Normal, Fire, Water, Grass, Electric,
31     Fighting, Poison, Ground, Flying
32 }
```

Listing 2: Clases principales del juego

10.3 Relaciones Principales

- SaveGames.UserID → Users.UserID
- Creatures.EvolvesTo → Creatures.CreatureID
- PlayerData.Team → Creatures.CreatureID